Chapitre 1 – Introduction à la programmation

Objectifs du chapitre

- Comprendre la notion d'algorithme et de variable.
- Découvrir les types de données de base en Python.
- Apprendre les opérations arithmétiques et logiques.
- Utiliser l'entrée/sortie standard avec `input()` et `print()`.

Variables et types de données

En Python, une **variable** est un nom associé à une valeur en mémoire.

- **Exemple :** `a = 5`
- **Types de base :**
  - `int` : entier (ex. 3, -5)
  - `float` : réel (ex. 3.14, -0.8)
  - `str` : chaîne de caractères (ex. `"Bonjour"`)
  - `bool` : booléen (`True` ou `False`)
- **Conversions :** `int()`, `float()`, `str()`

Exemple

```
a = 6
b = -5
c = 0.8
print(a, b, c)
```

Opérateurs arithmétiques et logiques

Opérateurs arithmétiques

- `+` : addition
- `-` : soustraction
- `*` : multiplication
- `/` : division
- `%` : modulo (reste de la division entière)
- `**` : puissance

Opérateurs logiques

- `and` : et logique
- `or` : ou logique
- `not` : négation

Exemple

```
a = 6
b = -5
c = 0.8

print((b > 0) or (c > 1))   # False
print(not (c < a))          # True
```

Entrées / sorties avec `input()` et `print()`

- `input()` lit une chaîne saisie au clavier.
- `print()` affiche des informations à l'écran.

Exemple

```
nom = input("Entrez votre nom : ")
age = int(input("Entrez votre âge : "))

print("Bonjour", nom)
print("Vous avez", age, "ans.")
```

Exercices pratiques – Chapitre 1

1. Écrire un programme qui **échange deux variables** `A` et `B`.
2. Écrire un programme qui **calcule le produit de 4 entiers**.
3. Écrire un programme qui **affiche la somme, la différence, le produit et la division** de deux nombres saisis par l'utilisateur.

Suivant →

Chapitre 2 – Structures conditionnelles

Objectifs du chapitre

- Utiliser les instructions conditionnelles `if`, `elif`, `else`.
- Manipuler les comparaisons et les tests logiques.
- Écrire des algorithmes décisionnels en Python.

Conditions simples et multiples

Syntaxe générale

```
if condition:
    instructions
elif autre_condition:
    instructions
else:
    instructions
```

Exemple : pair / impair

```
x = int(input("Entrer un nombre : "))

if x % 2 == 0:
    print("Pair")
else:
    print("Impair")
```

Exercices d'application – Chapitre 2

1. Vérifier si un nombre est **divisible par 7 et 17**.
2. Afficher le **nom du jour de la semaine** selon un nombre (1–7).
3. Déterminer la **catégorie d'un enfant** selon son âge.
4. Calculer une **facture de photocopies** selon un tarif dégressif.
5. Tester si une **année est bissextile**.
6. Écrire un **menu interactif** qui calcule la somme, le produit ou la moyenne de 3 nombres.

← Précédent
Suivant →

Chapitre 3 – Boucles et itérations

Objectifs du chapitre

- Savoir répéter des instructions avec `while` et `for`.
- Utiliser la fonction `range()`.
- Calculer des sommes, produits, factorielles et tables de multiplication.

Boucle `while`

La boucle `while` répète un bloc tant qu'une condition est vraie.

```
n = int(input("Entrer un nombre entre 1 et 3 : "))

while n < 1 or n > 3:
    n = int(input("Réessayer : "))
```

On utilise souvent `while` pour :

- Faire saisir une valeur valide.
- Répéter un traitement jusqu'à ce qu'une condition soit remplie.

Boucle `for` et fonction `range()`

La boucle `for` parcourt une séquence de valeurs.

```
for i in range(1, 11):
    print(i)
```

Exemples classiques :

- Affichage d'une **table de multiplication**.
- Calcul de la **somme** des entiers jusqu'à `n`.
- Calcul de la **factorielle** d'un nombre.

Exercices avancés – Chapitre 3

1. Demander un nombre et afficher les **10 nombres suivants**.
2. Afficher la **table de multiplication** d'un nombre donné.
3. Calculer la **somme des entiers** jusqu'à `n`.
4. Calculer `n!` (factorielle).
5. Trouver le **plus grand nombre** parmi une série d'entrées.
6. Simuler le **compte d'épargne de Mariam** (évolution du capital sur plusieurs années).

← Précédent
Suivant →

Chapitre 4 – Tableaux et matrices

Objectifs du chapitre

- Manipuler des tableaux (listes) en Python.
- Parcourir un tableau avec des boucles.
- Rechercher un élément dans un tableau.
- Trouver le maximum et son indice.
- Manipuler des matrices 2D simples.

Listes (tableaux) en Python

Une **liste** est une structure de données qui contient plusieurs valeurs.

```
T = [0, 5, -3, 10]
print(T[0])   # 0
print(T[1])   # 5
print(len(T)) # 4
```

- Les indices commencent à 0.
- `len(T)` donne la taille de la liste.

Saisir des valeurs dans un tableau

On peut remplir une liste avec une boucle `for` et la méthode `append()`.

```
T = []
for i in range(10):
    x = int(input("Entier n°{} : ".format(i+1)))
    T.append(x)
```

Cette technique sera utilisée dans plusieurs exercices du TD4.

Compter les valeurs selon un critère

Exemple : compter les entiers supérieurs ou égaux à 10.

```
compteur = 0
for x in T:
    if x >= 10:
        compteur += 1

print("Il y a", compteur, "valeurs ≥ 10.")
```

Cet exemple correspond à l'**Exercice 1 du TD4**.

Recherche d'un élément

Exemple : vérifier si un entier `V` se trouve dans le tableau.

```
V = int(input("Entrez la valeur à rechercher : "))
trouve = False

for x in T:
    if x == V:
        trouve = True

if trouve:


    print("V se trouve dans le tableau.")
else:
    print("V ne se trouve pas dans le tableau.")
```

Cet exemple illustre l'**Exercice 2 du TD4**.

Maximum et indice

On peut trouver le plus grand élément et son indice en parcourant le tableau.

```
max_val = T[0]
max_ind = 0

for i in range(1, len(T)):
    if T[i] > max_val:
        max_val = T[i]
        max_ind = i

print("Max =", max_val, "à l'indice", max_ind)
```

Ceci est utilisé dans les **Exercices 3 et 6 du TD4**.

Insertion et échange

Mettre une nouvelle valeur à un indice donné :

```
i = int(input("Indice à modifier : "))
val = int(input("Nouvelle valeur : "))
T[i] = val
```

Échanger les valeurs de deux indices :

```
i = int(input("Premier indice : "))
j = int(input("Deuxième indice : "))

tmp = T[i]
T[i] = T[j]
T[j] = tmp
```

Ces opérations correspondent aux **Exercices 4 et 5 du TD4**.

Moyenne de 6 entiers

Calculer la moyenne des éléments d'un tableau.

```
T = []
for i in range(6):
    x = int(input("Entier n°{} : ".format(i+1)))
    T.append(x)

s = 0
for x in T:
    s += x

moyenne = s / len(T)
print("Moyenne =", moyenne)
```

Cet exemple illustre l'**Exercice 7 du TD4**.

Matrices 3×3

On peut représenter une matrice par une liste de listes.

```
A = []
B = []
for i in range(3):
    ligneA = []
    ligneB = []
    for j in range(3):
        ligneA.append(int(input(f"A[{i}][{j}] = ")))
        ligneB.append(int(input(f"B[{i}][{j}] = ")))
    A.append(ligneA)
    B.append(ligneB)
```

Somme de deux matrices 3×3

Somme élément par élément des matrices A et B.

```
C = []
for i in range(3):
    ligneC = []
    for j in range(3):
        ligneC.append(A[i][j] + B[i][j])
    C.append(ligneC)

print("Matrice C =")
for ligne in C:
    print(ligne)
```

Cet exemple correspond à l'**Exercice 8 du TD4**.

Exercices pratiques – Chapitre 4

1. Saisir 10 entiers dans un tableau et compter ceux qui sont supérieurs ou égaux à 10.
2. Rechercher si une valeur V se trouve dans un tableau de 10 entiers.
3. Afficher l'indice du plus grand élément d'un tableau de 10 entiers.
4. Saisir un entier et un indice, puis insérer cet entier à l'indice donné.
5. Saisir deux indices et échanger les valeurs correspondantes dans le tableau.
6. Saisir 6 entiers, afficher le plus grand et calculer la moyenne.
7. Calculer la somme de deux matrices 3×3 saisies par l'utilisateur.

← Précédent
Suivant →

Chapitre 5 – Procédures et fonctions

Objectifs du chapitre

- Définir des procédures en Python avec `def`.
- Définir des fonctions qui renvoient une valeur.
- Utiliser des paramètres pour rendre le code réutilisable.
- Appliquer ces notions sur des exemples classiques (PGCD, factorielle, TTC).

Procédures (fonctions sans valeur de retour)

Une **procédure** est une fonction qui ne renvoie pas de valeur (ou renvoie `None`).

```
def salutation():
    print("Salam Alikoum !")
```

Appel :

```
salutation()
```

Procédure avec paramètre

On peut passer une chaîne à afficher en paramètre.

```
def afficher_chaine(texte):
    print(texte)

afficher_chaine("Bonjour tout le monde")
```

Procédure : table de multiplication

Procédure qui affiche la table de multiplication d'un nombre N.

```
def table_multiplication(n):
    for i in range(1, 11):
        print(n, "x", i, "=", n * i)

table_multiplication(5)
```

Fonctions avec return

Une **fonction** renvoie une valeur avec l'instruction `return`.

```
def carre(x):
    return x * x

y = carre(5)
print(y)  # 25
```

On peut utiliser la valeur renvoyée dans d'autres calculs.

Fonction PGCD

Exemple de fonction qui renvoie le Plus Grand Commun Diviseur (PGCD).

```
def pgcd(a, b):
    while b != 0:
        a, b = b, a % b
    return a

x = int(input("a = "))
y = int(input("b = "))
print("PGCD =", pgcd(x, y))
```

Fonction factorielle

La factorielle d'un entier n (notée n!) est le produit des entiers de 1 à n.

```
def factorielle(n):
    f = 1
    for i in range(1, n + 1):
        f *= i
    return f

n = int(input("n = "))
print(n, "! =", factorielle(n))
```

Fonction prix TTC

Calculer le prix TTC à partir du prix HT et d'un taux de TVA.

```
def prix_ttc(prix_ht, tva):
    return prix_ht * (1 + tva)

p = float(input("Prix HT = "))
t = float(input("TVA (ex : 0.2 pour 20%) = "))
print("Prix TTC =", prix_ttc(p, t))
```

Procédures et fonctions – Résumé

- `def nom(...)`: pour définir une procédure ou une fonction.
- Les paramètres permettent de réutiliser le même code pour différentes valeurs.
- `return` renvoie une valeur au programme appelant.
- Procédures : se concentrent sur l'affichage ou des actions.
- Fonctions : se concentrent sur un calcul et renvoient un résultat.

Exercices pratiques – Chapitre 5

1. Écrire une procédure qui affiche le message « Salam Alikoum ! ».
2. Écrire une procédure qui affiche la chaîne de caractères passée en paramètre.
3. Écrire une procédure qui affiche la table de multiplication d'un nombre N.
4. Écrire une fonction qui calcule le PGCD de deux nombres.
5. Écrire une fonction qui calcule la factorielle d'un nombre.
6. Écrire une fonction qui calcule le prix TTC à partir de prixHT et tva.

← Précédent